

Apellido	Nombre	Legajo	Corrigió
Ejercicio 1:	Ejercicio 2:	Ejercicio 3:	Total:

Ejercicio 1 (5 puntos).

Defina una clase **ParcialArboles** que contenga el siguiente método:

```
public static List<Integer> caminoParidadAlternante(GeneralTree<Integer> arbol)
```

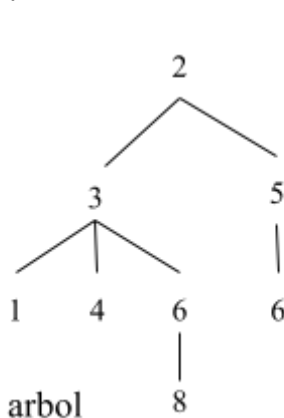
que recibe un árbol general de valores enteros y devuelve una lista con los valores de los nodos del **camino más largo** (máxima cantidad de nodos) desde la raíz hasta una hoja tal que los valores **alternen** entre par e impar en cada paso. Esto significa que cada nodo del camino debe tener paridad distinta a la de su padre (por ejemplo, si un nodo es par, su hijo inmediato debe ser impar y viceversa).

Si existen varios caminos con la misma longitud máxima que cumplen con la alternancia de paridad, se debe devolver el **primer camino** encontrado en un recorrido de izquierda a derecha.

En caso de no existir ningún camino desde la raíz hasta una hoja que cumpla con esta condición, el método debe devolver una lista vacía.

Nota: se considera al número 0 como par

Ejemplo



1. $2 \rightarrow 3 \rightarrow 4$
Par $\rightarrow$ Impar $\rightarrow$ Par ✓ Alternancia válida. Longitud = 2
2. $2 \rightarrow 5 \rightarrow 6$
Par $\rightarrow$ Impar $\rightarrow$ Par ✓ Alternancia válida. Longitud = 2
3. $2 \rightarrow 3 \rightarrow 1$
Par $\rightarrow$ Impar $\rightarrow$ Impar ✗ (No alterna par/impar)
4. $2 \rightarrow 3 \rightarrow 6 \rightarrow 8$
Par $\rightarrow$ Impar $\rightarrow$ Par $\rightarrow$ Par ✗ (No alterna par/impar)

- Ambos caminos 1 y 2 cumplen con la alternancia entre par e impar.
- Ambos tienen la misma longitud.
- En un recorrido de izquierda a derecha, el camino $2 \rightarrow 3 \rightarrow 4$ es el primero que se encuentra y el que tiene que retornar el método.

Observación importante:

- La alternancia puede comenzar desde cualquier valor en la raíz, ya sea par o impar. Lo importante es que a partir del segundo nodo (el primer hijo), los valores deben alternar la condición respecto al valor del nodo padre.
 - Por ejemplo, si la raíz del árbol fuera 3 (impar), un hijo con valor 2 (par) permitiría un inicio de camino alternante válido.

Tenga en cuenta que:

1. No puede agregar variables de instancia ni de clase a la clase **ParcialArboles**.
2. Debe respetar la clase y la firma del método indicado.
3. Puede definir todos los métodos y variables locales que considere necesarios.
4. Todo método que no esté definido en la sinopsis de clases debe ser implementado.

Ejercicio 2 (1.5 puntos)

Seleccione una opción. Sólo hay una respuesta correcta.

a.- Evalúe la siguiente expresión postfija: $4 \ 2 \ - \ 5 \ * \ 10 \ 5 \ + \ 8 \ 6 \ + \ 9 \ - \ / \ +$ y determine cuál es el resultado

- (a) 10
- (b) 12
- (c) 13
- (d) 14
- (e) 15

b. ¿Qué propiedad debe tener un heap para que el Heapsort ordene los datos ascendentemente?

- (a) Debe ser un árbol binario completo
- (b) Debe ser un árbol binario lleno
- (c) Debe ser una Max-heap
- (d) Debe ser una Min-heap
- (e) Puede ser Max-heap o Min-heap indistintamente

c.- Considere las siguientes sentencias:

- (i) Un árbol binario puede contener al menos 2^L nodos en el nivel L
- (ii) Un árbol general completo de altura H y grado K, contiene K^L nodos en cada nivel L, hasta la altura (H-1)
- (iii) La altura de un árbol binario lleno puede expresarse como $H = \log_2 (N+1) - 1$, siendo N el número total de nodos
- (iv) Un árbol general lleno de grado K y altura H, tiene $k^H - 1$ nodos

¿Cuál de las opciones es correcta?

- (a) (i) y (ii)
- (b) (i), (ii) y (iii)
- (c) (ii) y (iii)
- (d) (ii), (iii) y (iv)

Ejercicio 3 (3.5 puntos)

a.- Construya una Max-Heap con las siguientes claves usando el algoritmo de **Build-Heap**, muestre cada uno de los pasos seguidos para construirla

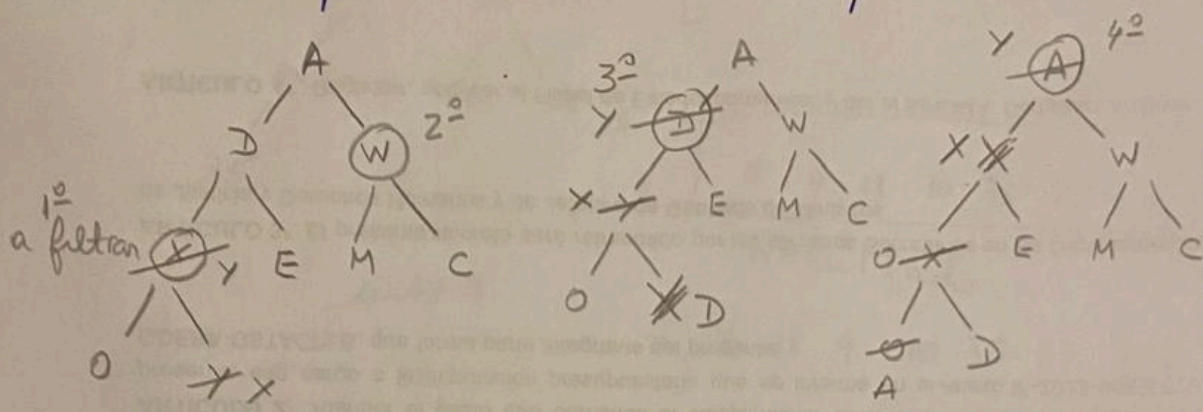
A D W X E M C O Y

b.- En la Max-Heap, construida anteriormente, ejecute una operación de DeleteMax.

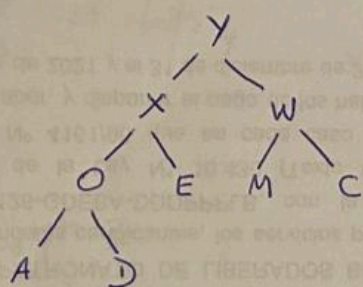
Ejercicio 3

Tema 1

a) Max-Heap usando Build Heap



Final:



b)

